

Cache-Friendly by Design: The Optimizations Inside cMVBT-OSIC

Amir Tonta

August 2026

Abstract

This document explains, layer by layer, how cMVBT-OSIC is built to keep a CPU’s caches, TLB, and lock-free fast paths working *for* it rather than against it. It is organized by architectural layer — physical page layout, then concurrency control built on top of that layout, then structural maintenance, then the transaction layer, then garbage collection, then the cross-cutting sharding and snapshot-caching machinery, then durability — because that is the order in which each optimization actually depends on the one before it, not the order in which any of it was written. A short section up front (§2) argues for four specific decisions that are *foundational*: nearly everything described later either depends on one of them for correctness, or depends on one of them for the cost it removes to exist in the first place. Wherever the implementation contains a controlled before/after measurement, that number is reported with its workload and caveats attached; wherever it was practical to do so, those numbers were independently re-measured while writing this document, and both are reported.

Contents

1	Introduction: cache-friendliness as the throughline	3
2	Foundational optimizations: what everything else stands on	3
2.1	Append-only, publish-once physical pages	4
2.2	Optimistic lock coupling and the SmartCell latch	4
2.3	OSIC: decoupling visibility from durability	4
2.4	A per-worker identity substrate	4
2.5	A related, second-order optimization: bounding version growth	4
3	Physical layer: pages, records, and memory layout	5
3.1	Append-only leaves and the version tuple	5
3.2	Structure-of-arrays leaf layout	5
3.3	The leaf’s live/validity mask	5
3.4	The internal page’s live mask	6
3.5	Exact page-sized allocation, payload indirection, and the hot/cold split	7
3.6	Same-allocation reuse across page types	7
3.7	Measured: leaf layout, isolated	7
4	Concurrency control: optimistic lock coupling	7
4.1	The SmartCell latch	7
4.2	Only the node being mutated is ever latched	8
4.3	Removing fences that atomics already subsume	8

5	Structural maintenance: choosing where to split and what to merge	8
5.1	Capacity and overflow thresholds	8
5.2	Split: a tiered boundary search	8
5.3	Merge: emptier-sibling selection and survivor accounting	9
6	Root indexing strategies	9
7	The transaction layer	10
7.1	Bounding version growth: self-overwrite and the delete/reinsert cycle	10
7.2	Caching resolved roots across a transaction’s lifetime	12
7.3	Inline bookkeeping: <code>SmallVec</code> instead of a pointer chase	12
7.4	Abort: reverse order, batched under one latch	13
7.5	The auto-commit fast path for single-key operations	14
8	Garbage collection: sharded, lock-free reclamation	15
8.1	Liveness driven directly off per-worker state	15
8.2	Own-shard-first reclaim with round-robin stealing	15
8.3	Historical routing keeps old readers off the hot reclaim path	15
9	Sharding as a system-wide pattern	16
9.1	Tables as shards	16
9.2	Cache-padded, per-worker atomic slots	16
10	Per-thread identity and the snapshot cache	17
10.1	Thread-local worker assignment	17
10.2	The snapshot cache: structure-of-arrays visibility memoization	17
11	OSIC and the write-ahead log	18
11.1	Instant commit, lazily hardened	18
11.2	Sharding the write channel	19
11.3	A lock-free alternative backend and per-thread batching	19
11.4	Ticket-less logging and exact framing sizes	20
12	Range scans and analytical terminals	20
12.1	Cursor-based internal routing	20
12.2	Zero-copy streaming terminals	21
12.3	Testing the cheap predicate first	21
12.4	Monomorphizing the visibility check	22
13	Workload and benchmark-harness optimizations	22
13.1	Fast, thread-local randomness	22
13.2	The alias-table Zipf sampler	22
13.3	Rank scrambling: a realism fix, not a speedup	23
13.4	Other workload-generation optimizations	24
14	Case study: trading contention for scan cost	24
14.1	The problem: two tiny tables dominate contention	24
14.2	The fix: an independent, tunable leaf capacity	24
14.3	Why the trade-off has a floor and a ceiling	25
14.4	The measured middle ground	25
15	Fresh measurements, independently reproduced	25
15.1	Leaf layout, reproduced on different hardware	25

15.2 Write-ahead-log backend throughput, reproduced with a fuller sweep	26
16 Combined measured gains at a glance	26
17 Conclusion	27

1 Introduction: cache-friendliness as the throughline

Almost every optimization in this system reduces to one of a small number of recurring moves: touch fewer cache lines to decide the same thing, avoid an allocation on a path that runs millions of times a second, let a lock-free reader skip a lock entirely, or shard a piece of shared bookkeeping so unrelated threads stop invalidating each other’s cache lines. This document is organized so those moves are visible as patterns, not as a list of unrelated tweaks.

Roadmap.

1. **Foundational optimizations** (§2): the four decisions everything else either depends on or is a response to.
2. **Physical layer** (§3): how one page holds several versions of a key, laid out to minimize cache and TLB pressure.
3. **Concurrency control** (§4): optimistic lock coupling and the latch every node uses.
4. **Structural maintenance** (§5): splitting and merging pages as they fill up or empty out.
5. **The transaction layer** (§7): bounding physical growth, caching resolved roots across repeated reads, reverse-order batched abort, and the auto-commit fast path for single-key operations.
6. **Garbage collection** (§8) and **sharding** (§9): reclaiming retired pages, and the recurring pattern of partitioning shared bookkeeping by worker or by table.
7. **Per-thread identity and the snapshot cache** (§10): how a worker remembers who it is and memoizes visibility decisions across calls, in detail.
8. **OSIC and the write-ahead log** (§11): decoupling visibility from durability, and the write-path optimizations that follow from it.
9. **Range scans** (§12), **workload/benchmark-harness optimizations** (§13), and a **case study** (§14) on trading contention for scan cost.
10. **Fresh measurements** (§15), a **combined gains table** (§16), and the **conclusion** (§17).

Evidence levels. Every claimed gain is tagged **Measured** (a before/after benchmark exists, with its workload and caveats), **Bounded** (a countable cost — an allocation, a lock, a cache line, a retry — is removably bounded, but no end-to-end percentage was recorded), or **Structural** (the change is primarily structural: it removes a class of cost or a piece of shared state rather than claiming a specific speedup). Combined benchmark results are never attributed to one component in isolation unless a measurement exists that isolates it.

2 Foundational optimizations: what everything else stands on

Four design decisions sit underneath nearly everything described later, in the specific sense that removing any one of them would not just cost throughput — it would make a later optimization either unsound or pointless.

2.1 Append-only, publish-once physical pages

Every leaf page is a fixed-capacity array that is only ever appended to. An obsolete, aborted, or deleted entry is *marked* dead; it is never physically overwritten or removed until the next structural-maintenance pass rebuilds the page from scratch (§3). Internal pages follow the same rule: a superseded child entry is never mutated, only shadowed by a newer appended entry (§3, §8).

What this underwrites. Optimistic lock coupling (§4) lets readers inspect a page with no latch at all specifically because nothing they might observe was ever mutated after being published. The internal-page live mask (§3) is a pure function of *disjoint, append-ordered intervals* — it has no meaning if slots could be mutated in place. And the version-growth bound in §7 rests on the notion of “this transaction’s own, exclusively owned tuple,” which only exists because writes append rather than clobber.

2.2 Optimistic lock coupling and the SmartCell latch

Readers take an atomic version sample of a node’s latch and validate it after use, with no lock acquired at all; writers upgrade to an exclusive, seqlock-style write lock only on the node they actually mutate (§4).

What this underwrites. Every sharding optimization in §9 — per-worker bookkeeping slots, the garbage collector’s per-shard dead-block index, cache-padding those slots against false sharing — is a fix for contention *among concurrent, lock-free readers and writers*. If reads instead took a page latch, most of that contention would already be serialized by the latch, and sharding the bookkeeping around it would buy nothing.

2.3 OSIC: decoupling visibility from durability

Every transaction draws a start and a commit timestamp from one global counter; a write becomes visible to other workers the instant its commit timestamp lands in the committing worker’s append-only commit log — an in-memory step with no wait on disk I/O (§11).

What this underwrites. The whole write-ahead-log performance story in §11 — sharding the write channel, a lock-free alternative backend, exact framing sizes — is optimizing the cost of *catching up durability asynchronously*. If commit instead required a synchronous disk round-trip, none of those micro-optimizations would matter: the round-trip’s latency would dwarf all of them by one to two orders of magnitude.

2.4 A per-worker identity substrate

A worker identity is assigned once per (tree, thread) and cached thread-locally (§10); it is the index into every per-worker structure the rest of the system builds (§9).

What this underwrites. Sharding, as a pattern, needs something to shard *along*. The garbage collector’s dead-block tracker, the transaction context’s live-snapshot slots, and the thread-local snapshot cache are all indexed the same way. Without a stable per-worker identity assigned once and reused, each of those would need its own separate discovery mechanism.

2.5 A related, second-order optimization: bounding version growth

One further optimization is worth flagging here even though it depends on the four foundations above rather than being depended on by everything: without the version-growth bound described in §7, the structural-maintenance tuning in §5 — capacity thresholds, split-boundary search, merge-candidate selection — would be fighting an unbounded adversary, since a single

transaction that writes one key many times inside itself can otherwise leave a page with no valid split boundary at all.

3 Physical layer: pages, records, and memory layout

3.1 Append-only leaves and the version tuple

A leaf’s record array is append-only, as established above; a packed length counter tracks active vs. dead slots. Each tuple carries a version header: an insertion stamp (a packed (**worker_id**, **ts_start**), constant for the whole transaction, not per write) and an optional deletion stamp. Lookups scan *backward* for the newest live entry, since physical append order places the newest write last — this is exactly why point lookup is a reverse linear scan rather than a sorted search: the leaf is not key-sorted, and append order is the only order it actually guarantees.

3.2 Structure-of-arrays leaf layout

A leaf’s physical layout is a dense key region followed by a parallel version/payload region, not one array of complete {key, version, payload} records:

key_region:	[key0, key1, key2, ...]
data_region:	[(version0, payload0), (version1, payload1), ...]

Point lookup walks the key region backward; version metadata and payload cache lines are touched only *after* a key matches, and a range scan can test its key range before loading the parallel data at all. This is a pure locality change: for u64 keys and a one-word payload slot, one physical entry is still 32 bytes (8 bytes key, 24 bytes version+payload), so capacity is unaffected — only how many unrelated cache lines a rejected key touches on the way to being rejected.

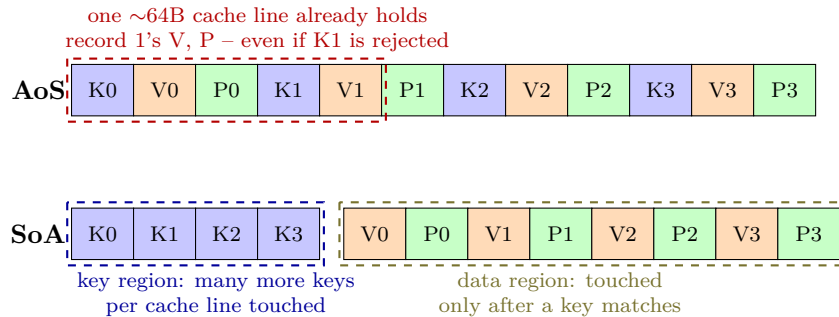


Figure 1: Array-of-structs (top) interleaves one record’s key, version, and payload, so rejecting a key still pulls its neighbors’ version/payload bytes into cache. Structure-of-arrays (bottom) separates the two regions, so a scan’s key-rejection pass touches only the key region, and the data region is paid for exactly once per match, never per rejection.

3.3 The leaf’s live/validity mask

A normal leaf has at most 123 physical slots, so two 64-bit words (128 bits) cover every slot’s liveness directly inside the page, next to the key/data regions themselves, rather than behind a separate boxed-slice handle and its pointer chase. A clear bit means the slot has been invalidated (by an abort or a compaction) and can be skipped by lookup *before* its version header and payload are ever touched — the same “reject cheaply, load lazily” principle as the structure-of-arrays layout above, taken one level further: instead of loading a key to reject a slot, a lookup can reject a whole slot with one bit test against a mask that already lives in the leaf’s own first cache line. This is a *physical*-liveness mask, distinct from MVCC visibility (which also depends on the

reading transaction’s snapshot and cannot be represented by one bit per slot), but it composes with the version check that follows it: a lookup pays for the version/visibility check only on slots this mask has not already ruled out. Deliberately oversized leaf variants (§14) fall back to a heap bitmap through a same-sized union representation, so the normal leaf’s mask never grows past two words no matter how large a specialized leaf variant’s capacity gets.

3.4 The internal page’s live mask

Internal pages are append-only across structural versions: replacing a child appends one or more new key-interval entries while the old interval stays physically present for older snapshots to route through. Deciding which entries are *currently* live — the set a fresh, non-historical traversal or a structural rebuild should actually consider — would naively require rebuilding a temporary liveness vector from every pair of entries’ intervals each time it is needed, an $O(\text{FAN_OUT}^2)$ reconstruction paid again and again on the structural-maintenance cold path.

The internal page instead carries its own live mask, maintained incrementally *at append time*: appending a new child interval clears the mask bit of every older entry that interval overlaps, sets the new entry’s own bit, and publishes the new physical length as usual. This exploits an invariant that holds throughout the tree’s life — live sibling key intervals are always pairwise disjoint — so “is entry i still part of the current partition” reduces to one word load and one bit test, and counting the live entries reduces to a popcount over at most two words. Every later consumer of this information — merge-candidate selection, survivor counting, split copying — reads the mask directly instead of rebuilding it, moving the $O(\text{FAN_OUT}^2)$ cost from “every time someone asks” to “once, when the page’s contents actually change.”

This mask is deliberately *not* consulted by historical routing: a child superseded in the current partition may still be the correct route for an older snapshot, so historical reads keep selecting via each entry’s own immutable version, independent of this mask entirely. The mask is a current-state acceleration structure layered on top of the version metadata, not a replacement for it.

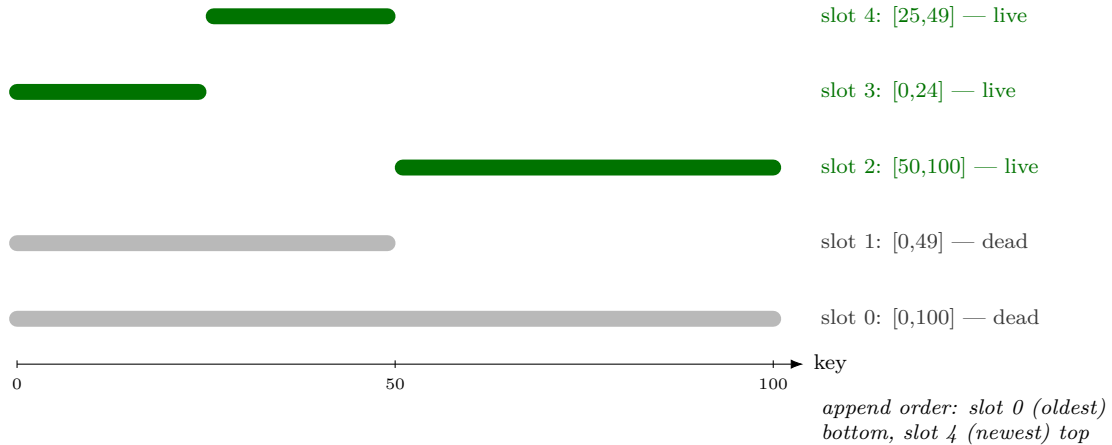


Figure 2: Five entries appended over two splits of the same page. Slot 0’s interval is entirely covered by the newer slots 1–2, so it is dead; slot 1’s interval is entirely covered by the still-newer slots 3–4, so it too is dead, even though it is younger than slot 2. The live mask marks exactly {2, 3, 4}: the newest layer of pairwise-disjoint intervals that together cover the whole key range, derived once when slots 3–4 are appended, not recomputed by whoever asks later.

3.5 Exact page-sized allocation, payload indirection, and the hot/cold split

The 4 KiB target applies to the full allocated cell, including the concurrency latch’s own version word and node metadata — not just the record array. With `FAN_OUT = NUM_RECORDS = 123`, the cell lands at exactly 4096 bytes; one additional record per array would cross that boundary. This two-record trim is **Measured**: page-aligned allocations rose from 25% to 100% of the time, with a 9–12% reduction in dTLB load misses — concrete evidence that “nominally full” and “lands cleanly on a hardware page” are two different numbers, and only the second one is free.

A one-word payload slot stores a `u64` or a thin row handle inline; larger rows go through a reference-counted pointer instead, so cloning a read result or copying a survivor during a split increments a refcount rather than copying a whole row. The tree’s own top-level handle is split into a hot part — root, allocator, transaction context, everything the traversal-critical path touches — and a separately boxed cold part holding construction-time configuration (WAL handle, table id, capacity bounds) that the hot path never needs to read, purely to shrink the cache footprint of the part that does.

3.6 Same-allocation reuse across page types

A node’s leaf and internal representations occupy one **union**: switching a node from one page type to the other (as structural maintenance sometimes requires) drops the previously active variant and constructs the newly needed one in the same allocation, rather than freeing one page object and allocating a fresh one of the other type. The allocator only ever sees a churn of same-sized objects, never a mixed-size alternation between two different page-type footprints.

3.7 Measured: leaf layout, isolated

The structure-of-arrays layout’s benefit was isolated in a standalone, dependency-free microbenchmark comparing it against the earlier array-of-structures layout, with no tree traversal, latching, transactions, or allocation in the timed region: 16,384 independently boxed leaf pages per layout (a 62.5 MiB working set each), five append-ordered update waves per key, point lookups walking backward, scans checking the range before visibility.

Operation	Array-of-structs	Structure-of-arrays	Speedup
Point: latest visible	30.57 Mlookups/s	33.17 Mlookups/s	1.085×
Point: 1-version fallback	18.43 Mlookups/s	20.15 Mlookups/s	1.093×
Point: 3-version fallback	9.03 Mlookups/s	11.02 Mlookups/s	1.221×
Scan: full leaves	322.84 Mslots/s	320.65 Mslots/s	0.993×
Scan: 20% key ranges	354.82 Mslots/s	389.09 Mslots/s	1.097×

Point lookups improve because the key region rejects non-matching entries without touching the version/payload cache lines at all, and the effect grows with how many dead versions a lookup has to walk past before finding a live one. Full scans are essentially unaffected because they touch every byte of the leaf regardless of layout. §15 reproduces this measurement independently and discusses a genuine hardware-dependence found while doing so.

4 Concurrency control: optimistic lock coupling

4.1 The SmartCell latch

Every node’s latch is a single 64-bit versioned word with three logical bit ranges packed into it: a seqlock-style write flag, pin-related flags, and a permanent retired tombstone bit. An optimistic read is one atomic load with the write/retired bits masked out — no lock taken, no cache line

written by a reader at all. Upgrading to exclusive is a compare-and-swap from the captured read version to the write-flagged version, failing immediately if the retired bit is already set. Unlock stores the incremented version with the retired bit preserved exactly as it was found, so a same-critical-section retirement is never clobbered by the unlock itself. A guard that will unconditionally be consumed with no back-out path can retire directly from a reader snapshot with a single compare-and-swap, skipping the intermediate exclusive-lock step entirely — one fewer lock acquisition on the structural-maintenance path for exactly the guards that never need to back out.

4.2 Only the node being mutated is ever latched

Readers are fully lock-free: they sample a node’s version, read its (immutable, or atomically published) contents, and validate the sampled version before relying on the result, retrying if it changed. Writers acquire an exclusive guard only on the node they are actually about to mutate — there is no global traversal guard, and no hand-over-hand latching down the tree. This is the property the whole sharding story in §9 exists to exploit: contention among lock-free participants is a real cost worth removing, in a way it would not be if a coarser lock already serialized most of it.

4.3 Removing fences that atomics already subsume

Once a data-race-free protocol’s own atomic loads/stores are trusted to carry the ordering a memory fence would otherwise provide, a standalone fence next to that atomic op is pure overhead. Several such fences, added early out of caution, were identified as redundant once the protocol’s actual happens-before edges were traced precisely: a length publish already carries the ordering a paired fence next to it would duplicate; a page’s soft-commit counter can use a relaxed store because the write-unlock that follows it always publishes with an acquire/release pair of its own; a node’s type tag, mutated only under the exclusive write lock, needs no fence on drop because the latch’s own release already publishes it. The general pattern: every real publication edge in this system is exactly one atomic operation wide, and a fence is only ever needed where no atomic operation already sits on that edge — which, once the protocol is fully traced, turns out to be almost nowhere on the per-record hot path.

5 Structural maintenance: choosing where to split and what to merge

Splitting and merging pages is the cold path that keeps every page’s capacity bounded; the algorithms below decide *where* to cut and *which* neighbor to combine with, so that the common case needs no retry.

5.1 Capacity and overflow thresholds

An internal page is allowed to fill much closer to its true capacity before a split is attempted at all, rather than reserving several slots of headroom defensively — headroom that is rarely needed and, when it is not, simply forces more structural work than the page’s real capacity requires.

5.2 Split: a tiered boundary search

Splitting a leaf chooses between compacting dead versions out of the page in place (when that alone brings it back under capacity) and cutting it into two pages at a key boundary. Choosing that boundary is a search with two competing goals: prefer a boundary that does not cut across

one key’s own version chain, and prefer a boundary whose two halves both fit within capacity. Rather than combining both goals into one greedy pass — which can miss a boundary that satisfies both simply because it lies farther from the naive midpoint than the search first looks — the search tries a small number of increasingly permissive tiers in order: first, a boundary that satisfies both goals; failing that, one that fits capacity but may cut a version chain; failing that, one that avoids cutting a chain but may leave one side over capacity; and only as a last resort, the raw midpoint. This tiered search costs a few extra integer comparisons in the rare case where the ideal boundary is not the first one tried, in exchange for finding a boundary that satisfies both goals whenever one genuinely exists.

5.3 Merge: emptier-sibling selection and survivor accounting

Merging considers *both* adjacent siblings and picks whichever has more available room, rather than always defaulting to one side — a page with a nearly full sibling on one side and a nearly empty one on the other should combine with the empty one. Deciding whether a merge will actually fit requires knowing how many records will genuinely survive the combination, not merely how many are currently counted *active*: a record whose deleting transaction is still unresolved must be retained until that transaction’s fate is known (it may still abort and need its predecessor back), so an active-count estimate alone can understate the true survivor count and risk overflowing the fixed-capacity destination page. The merge path counts true survivors before committing to the merge, falling back to a key split instead whenever the honest count would not fit — the same preflight philosophy as the leaf’s live/validity mask: decide cheaply and correctly before doing the expensive copy, not after.

When combining two leaves’ records into one, physical append order is preserved rather than re-sorted by any transaction timestamp: the newest-version lookup described in §3 depends on physical append order tracking recency, and re-sorting by an unrelated key would silently break that invariant for every future lookup on the combined page.

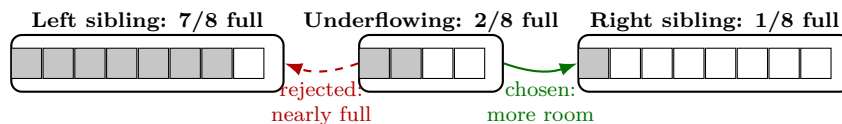


Figure 3: The underflowing center page considers both adjacent siblings rather than defaulting to one side. The left sibling is nearly full and is rejected; the right sibling has more room and is chosen — the merge that actually has a chance of succeeding, decided before either candidate is latched for the real copy.

6 Root indexing strategies

Before traversing anything, a reader has to find “the root as of my snapshot” — a lookup keyed by version into a table’s own root-version index. Four interchangeable structures are available for this, each trading lookup complexity against per-append cost and memory overhead differently:

Strategy	Structure	Lookup	Notes
Vanilla	Linked list of roots	$O(n)$ linear	Simplest baseline.
Skip-list	Lock-free ordered map	$O(\log n)$ expected	Concurrent inserts and lookups with no shared lock.
B-tree-indexed	An external, latch-coupled B+-tree keyed by version	$O(\log n)$	Reuses a full concurrent index just to look up root pointers by version.
Frugal (default)	A lock-free singly-linked list with a probabilistic skip tower	Sub-linear expected, flat-chain worst case	Appending is a single pointer store with no retry loop needed, since appends are already serialized by the writer that is publishing the new root; a much smaller per-entry footprint than a full skip-list tower.

Each of these is one single, tree-wide structure holding every historical root — it is not partitioned the way §9’s sharding pattern partitions other state, because the thing being optimized here is different: the cost of resolving one specific version to one specific root pointer, not contention among many workers touching unrelated pieces of shared bookkeeping.

7 The transaction layer

7.1 Bounding version growth: self-overwrite and the delete/reinsert cycle

This is one of the more interesting problems this system solves, and it is worth walking through carefully, because the fix is small and the failure mode it prevents is not obvious until you look for it.

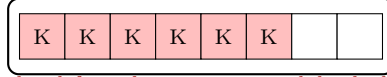
Every write mints one more physical record tuple, and an *uncommitted* tuple cannot be discarded (its transaction might still commit) or coalesced with anything (nothing has superseded it yet, since nothing else can even see it). A naive update path is simply “mark the old version dead, append the new one” on every call — which means a single open transaction that writes the same key N times before committing leaves N physical, uncommitted tuples for that one key, none of them reclaimable until the transaction resolves. If N grows large relative to a page’s capacity, the split-boundary search in §5 runs out of options entirely: a page dominated by one key’s own version chain has no key boundary to cut at all. A concrete, realistic trigger for this: a single order transaction that touches the same inventory row twice (two order lines for the same item) writes that row twice within one transaction.

The fix is a single equality check performed before falling through to the normal versioned path:

```
// Self-overwrite fast path: this record was already written
// by *this* transaction -- nobody else can possibly be
// relying on its current content, so mutate it in place.
if leaf_page.version_at(position).insertion_stamp() == stamp {
    leaf_page.set_payload_at(position, payload);
    return (CRUDOperationResult::Updated(stamp.ts_start()), false);
}

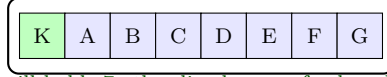
// Otherwise: normal versioned update (delete old, push new).
if !leaf_page.version_mut_at(position).delete(stamp) {
```

Without a bound: 6 writes to key **K**, one open transaction



no other key left in the page — no valid split boundary

With self-overwrite: the same 6 writes, the same transaction



page still holds 7 other live keys — freely splittable

Figure 4: The same transaction writing key **K** six times before committing. Without a bound (top), every write appends another uncommitted tuple for **K**, crowding out every other key the page might otherwise hold and leaving the split-boundary search in §5 with nothing to cut at. With self-overwrite (bottom), those same six writes mutate one tuple in place, so the page’s remaining capacity is exactly what it would have been if **K** had been written once.

```

    return (CRUDOperationResult::ZeroAffected(KeyAlreadyDeleted), false);
}
let current_len = leaf_page.len();
leaf_page.push_uncommitted(
    RecordPoint::new(key, VersionInfo::new(stamp), payload), current_len);
leaf_page.commit_delta(0, 1);
(CRUDOperationResult::Updated(stamp.ts_start()), true)

```

Detection is a single comparison, not a heuristic: a transaction stamp packs (*worker_id*, *ts_start*) and is constant for the *whole* transaction, so “was the newest live version of this key written by me, in this very transaction” reduces to one equality test. **Safety** follows directly from visibility: the found record is only reachable this way if it belongs to the same transaction, and nothing else can possibly be relying on the current content of a tuple nobody else can see yet. If the found record belongs to another transaction, or is already committed, the code falls through to the ordinary delete-then-append path unchanged, so this is a strict fast path with no change to external behavior.

The mirror-image case is a delete followed by a reinsert of the same key within one transaction. A delete must physically retain the predecessor (abort needs to be able to undelete it), so the first delete/insert pair in a transaction must append a genuine replacement tuple. But repeating that pair — delete, reinsert, delete, reinsert — would otherwise append another replacement on every cycle. The fix checks whether the newest record’s insertion *and* deletion stamps both belong to the current transaction, and if so, clears the deletion mark and replaces the payload in place instead of appending again:

```

if leaf_page.version_at(position).insertion_stamp() == stamp
    && leaf_page.version_at(position).deletion_stamp() == Some(stamp)
{
    leaf_page.version_mut_at(position).undelete();
    leaf_page.set_payload_at(position, payload);
    leaf_page.commit_delta(1, -1);
    return (CRUDOperationResult::Inserted(stamp.ts_start()), false);
}

```

Together, these two checks give a clean bound: N updates or delete/reinsert cycles on one key by one transaction cost at most *one* unresolved replacement tuple, not N . A transaction attempting to extend another transaction’s still-open chain on the same key is simply rejected (first-writer-wins), so this bound holds under concurrency too, not just within one thread. Once a writer resolves, ordinary compaction discards the obsolete committed versions, and pages containing

several distinct keys remain freely splittable at a key boundary — the version-growth problem this section opened with never gets a chance to occur.

7.2 Caching resolved roots across a transaction’s lifetime

Finding “the root as of my snapshot” for a given table requires a lookup into that table’s root-version index (§6). A transaction that reads or writes the same table more than once — the common case for anything beyond a single-statement operation — would otherwise repeat that lookup on every call, even though the answer cannot change: root histories are append-only, and once a snapshot’s root has been resolved, no later publication can change what that specific, already-fixed snapshot should see.

Each transaction keeps a small inline table of per-table state, one entry per table it has actually touched, each holding that table’s tree handle and (once resolved) its cached root for this transaction’s snapshot:

```
fn read_tree_and_root(&mut self, table: TableId) -> (Arc<Tree>, BlockRef) {
    let slot = self.table_slot(table);
    if let Some(root) = &self.tables[slot].read_root {
        return (self.tables[slot].tree.clone(), root.clone());
    }
    let tree = self.tables[slot].tree.clone();
    let root = tree.retrieve_root_for(self.ts_start);
    self.tables[slot].read_root = Some(root.clone());
    (tree, root)
}
```

The first read or write against a table resolves and caches its root; every subsequent operation against that same table, for the rest of the transaction, reuses the cached handle instead of re-walking the root index.

7.3 Inline bookkeeping: SmallVec instead of a pointer chase

The table-state list that root caching above lives in — and, as this subsection covers, the transaction’s write set, the database’s own table list, and a TPC-C-specific write set — are all the same container type: a **SmallVec**, which embeds its element buffer directly inside the surrounding struct up to a fixed inline capacity, and only falls back to a separate heap allocation if that capacity is ever exceeded. Below the inline capacity, correctness is unaffected by any of this — it behaves exactly like an ordinary growable vector, spilling to the heap transparently the moment it needs to. What changes is locality: a transaction is created, used, and dropped by exactly one worker thread, so keeping its own bookkeeping inline means that thread’s repeated touches to its own transaction stay inside the same handful of cache lines the transaction object itself occupies, instead of chasing a pointer out to a second, independently allocated buffer for what is, in the overwhelming majority of cases, a small, fixed-shape piece of state. This is the same locality principle behind the per-worker slots in §9 and the thread-local caches in §10, applied one level down, to a single transaction’s own working set rather than to a tree-wide array indexed by worker.

- **The per-table cache** (§7.2, capacity 8) holds one entry per table the transaction has actually touched — a tree handle and, once resolved, the cached root above. Eight entries comfortably covers an ordinary multi-table OLTP transaction; an unusually wide one simply spills. Because it is inline, checking “have I already resolved this table’s root” is a linear scan over a handful of contiguous entries sitting next to the transaction’s other fields, not a hash-map lookup and not a chase through a second allocation — for a list this short, that scan is not an approximation of a smarter data structure, it *is* the fast path.

- **The write set** (capacity 16) records every key this transaction has actually written, in the true order it wrote them, and is exactly what abort (§7.4, below) walks in reverse. An ordinary transaction’s writes stay inline for the whole transaction, so both the writes themselves and the eventual abort walk over them touch memory local to the transaction object rather than a second buffer allocated and then immediately torn back down.
- **The database’s own table list** (capacity 256) stores every table’s name and tree handle. Tables are created once, rarely, at database setup, so the realistic table count is always far below this capacity in practice. The benefit here is subtly different from the two cases above: the table list is reached through one shared, atomically published handle (§9), and a plain growable vector behind that handle would need *two* heap indirections to reach a given table — one to reach the vector’s own header, a second to follow the header’s separate buffer pointer to the actual entries. An inline buffer sits inside the same allocation as the header, which is itself already inside the shared handle’s own allocation, so resolving a table costs exactly one indirection, not two, for as long as the table count stays under 256.
- **A TPC-C-specific write set** (capacity 24) exists because TPC-C writes to two structurally independent tree configurations (§14) within one logical transaction, and abort’s reverse ordering has to reflect the transaction’s true, combined write order across *both* configurations, not two independently-reversed per-configuration lists — so this write set is deliberately one single inline list spanning both, sized generously above this workload’s typical transaction shape rather than reused verbatim from the general-purpose write set above.

None of these capacities are load-bearing for correctness — they were each chosen to comfortably cover the common case for what uses them, with the ordinary, non-catastrophic fallback of a real heap allocation on the rare transaction or table catalog that actually needs more.

7.4 Abort: reverse order, batched under one latch

Reverting a transaction’s writes is not one atomic step across every key at once — each individual reversal re-exposes that one key to any other concurrent transaction the instant it completes, before the rest of the abort has finished. This makes the *order* of reversal a correctness question, not just a performance one: if an earlier write in the transaction effectively acted as a gate that a later write depended on (for example, removing a queued work item before updating the records that item referred to), reverting in forward chronological order would re-expose that gate *first*, while several of its dependents are still mid-reversal — a concurrent reader could observe the gate reopened and race the transaction’s own in-progress rollback of the state it gated. Reverting in reverse (last write first) avoids this: by the time the gating write’s own reversal makes it visible again, everything the transaction did after it has already been fully undone.

The write set is walked back to front, and adjacent writes to the *same* key are collapsed into a single call that reverts all of them in one leaf traversal and one write latch, rather than one traversal per version:

```
let mut end = self.written.len();
while end != 0 {
    let (slot, key) = self.written[end - 1];
    let mut start = end - 1;
    while start != 0 && self.written[start - 1] == (slot, key) {
        start -= 1;
    }
    self.tables[slot].tree.abort_writes(key, stamp, end - start);
    end = start;
}
```

Since the version-growth bound in §7.1 already keeps repeated writes to one key inside a transaction as a short, physically adjacent run in the write set, this batching is not a rare case: it is the direct consumer of that bound, turning “however many times this transaction touched this key” into one latch acquisition instead of one per touch.

A transaction that ends without ever writing anything (a read-only transaction, or one whose only attempted write returned `Conflict`) releases its snapshot directly, without drawing a commit timestamp or appending a commit-log entry at all — there is nothing to make visible and nothing to revert, so both the commit and abort paths degenerate to the one step that is actually needed.

7.5 The auto-commit fast path for single-key operations

Alongside multi-statement transactions, the tree exposes single-key operations that commit themselves: an auto-committed insert, update, or delete is not a snapshot transaction with a one-entry write set, it is a shorter protocol tailored to exactly one key. Its write path, for an insert:

```
let leaf_guard = self.traversal_write_olc(key);
// ... find newest version, check visibility, decide ...
let stamp = self.wal_start_commit(|_| CRUDOperation::Insert(key, payload.clone()));
leaf_page.push_uncommitted(RecordPoint::new(key, VersionInfo::new(stamp), payload), len);
leaf_page.commit_delta(1, 0);

// Commit while the leaf is still write-locked. OLC readers cannot
// validate the new prefix until the guard is released, so the version
// is already in OSIC's commit log at the first instant it can be
// observed successfully.
let ts_commit = self.commit_tx(stamp.worker_id());
drop(leaf_guard);
self.wal_log_commit(stamp, ts_commit);
```

The order of the last three steps is the entire point: the commit-log append happens *before* the leaf guard is released, not after. Since a lock-free reader cannot successfully validate a leaf while it is write-locked, the earliest possible instant any reader could observe the new record is strictly after the commit-log entry already exists — closing what would otherwise be a window where a reader sees a published record whose commit has not yet landed. Releasing the WAL’s durability record is left for afterward, deliberately: visibility and durability are separate guarantees here (§11), so the caller is never blocked on disk I/O to get a result back.

An auto-committed write also does not register a full OSIC snapshot the way a transaction’s `begin` does: it draws a start timestamp for the record’s own stamp and a commit timestamp for the commit log, but never publishes the start timestamp as a long-lived, GC-visible reader — it does not retain a root, a page, or a timestamp after the call returns, so there is nothing later that a full snapshot registration would need to protect. Protection during the traversal itself comes from a lighter-weight reclamation pin rather than a full snapshot, sized to exactly the guarantee this shorter-lived operation actually needs.

A closely related fast path lets a caller atomically read-and-patch a single field of an existing row while holding the very same leaf write latch used for the write itself, deriving the replacement payload from the current one in one guarded step — so a single-field update never has to materialize, mutate, and write back an entire row, and never risks losing a concurrent updater’s change to a different field of the same row, because both are serialized by the one latch the whole operation runs under.

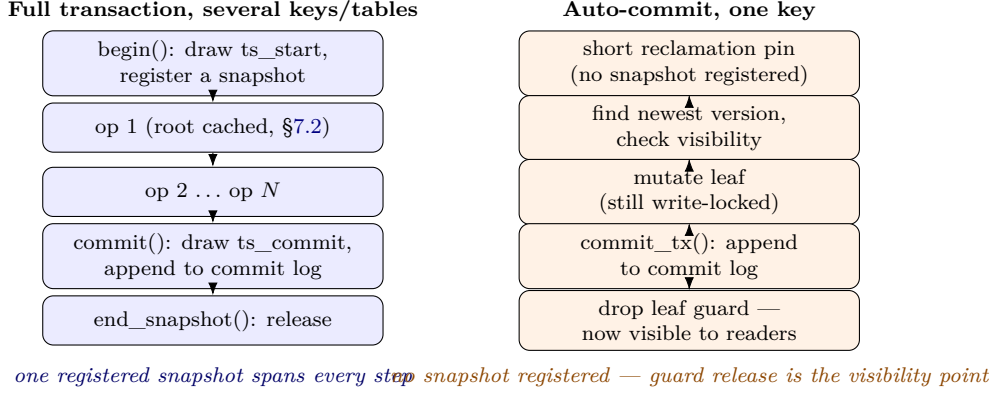


Figure 5: A full transaction (left) pays for a registered snapshot once, up front, and amortizes it across however many operations it goes on to perform — exactly what makes root caching (§7.2) and batched abort (§7.4) worthwhile. A single auto-committed operation (right) never needs that amortization, so it skips snapshot registration entirely: a short reclamation pin protects the traversal instead, and the leaf guard’s own release — after the commit-log append, not before — is the operation’s entire visibility and durability handoff.

8 Garbage collection: sharded, lock-free reclamation

8.1 Liveness driven directly off per-worker state

Deciding whether a retired page can be reused reduces to one question: is there any live reader whose snapshot might still need it? Rather than maintaining a separate index of active transactions to answer this, liveness is read directly off the same per-worker snapshot slots described in §9 — the oldest currently published snapshot across all workers is the answer, computed with a plain scan over a small, fixed number of per-worker slots rather than a query into a second, independently-maintained structure that could disagree with the first.

8.2 Own-shard-first reclaim with round-robin stealing

Retired pages are indexed in one shard per worker rather than one shared structure: a page a worker retires is registered only into that worker’s own shard, and a worker looking for a page to reuse checks its own shard first, falling back to a shared round-robin cursor across other shards only when its own is empty. Since a reclaiming worker only needs *a* reusable page, not the globally oldest one, trading strict global reuse order for per-worker insertion locality is a clean win: the common case touches only the calling worker’s own shard and its own cache lines, and the fallback path only runs when that shard genuinely has nothing to offer.

8.3 Historical routing keeps old readers off the hot reclaim path

An older reader is never the reason a rebuild has to copy a record forward into a replacement page: instead, that reader continues to route through the retired source block via the historical root/version chain until it finishes, at which point the block becomes reusable on its own. This keeps the decision “does this specific replacement page need to retain this record” local to what structurally-current readers and possible aborts actually need, rather than coupled to whatever the globally oldest reader anywhere happens to be doing.

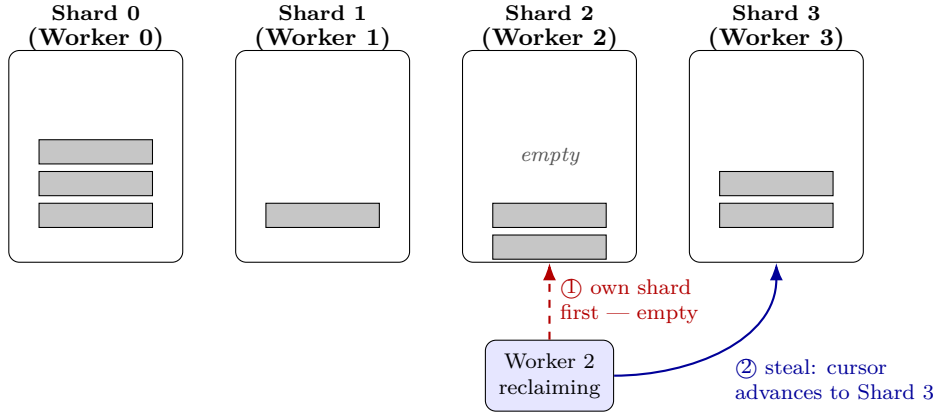


Figure 6: Reclaiming worker 2 checks its own shard first (Shard 2, empty here) before falling back to a shared round-robin cursor that advances to the next shard (Shard 3) rather than always re-scanning from Shard 0 — so no single remote shard is favored repeatedly, and the common case (a worker’s own shard already has something) never touches this fallback path at all.

9 Sharding as a system-wide pattern

Sharding, in this codebase, is not horizontal key-range partitioning of one index. It is the same move applied four times: take a piece of hot, shared bookkeeping state and partition it by worker (or, in one case, by table) so that unrelated threads stop contending on the same cache lines.

9.1 Tables as shards

Each logical table gets its own physically separate tree, its own root index, and its own dead-block tracker, rather than several tables being multiplexed into one shared tree. Concurrent transactions touching different tables therefore never contend on one tree’s root or internal-page latches, or on one shared dead-block index. The table list itself is published behind a lock-free read-copy-update handle: readers resolve a table with one atomic load and no lock at all, and adding a new table — a rare, cold-path operation — builds a new table list and swaps it in without ever blocking a concurrent reader. What must stay global for cross-table snapshot isolation — the shared clock, commit-log pruning, and snapshot liveness — lives in one shared, per-database context rather than being duplicated per table.

9.2 Cache-padded, per-worker atomic slots

Each worker’s currently published snapshot, its transaction-nesting depth, and a conservative in-flight registration bound each live in their own array, one slot per worker, written only by that worker and read with a simple scan by anyone computing the oldest live snapshot. This directly replaced an earlier design built around shared structures — a shared ordered map of active snapshots, and a single shared counter tracking in-flight registrations — both of which turned into serialization points precisely because every worker had to touch the same shared location on every registration and every liveness check.

Because adjacent workers’ slots in the same array can otherwise share one 64-byte cache line, each of these three per-worker arrays pads every entry out to its own cache line. Without that padding, one worker updating its own logically-independent slot would still invalidate a neighboring worker’s cached copy of *their* slot purely because the two happen to share a line — a classic false-sharing cost that grows directly with core count, and one that a purely single-writer-per-slot design can otherwise look immune to on paper while still paying for it in practice.

10 Per-thread identity and the snapshot cache

This section covers, in detail, how a thread knows who it is on a given tree and how visibility checks against *other* workers are memoized — the mechanism that the sharding pattern above (§9) depends on having in the first place.

10.1 Thread-local worker assignment

A worker identity is a small integer, handed out once per (tree, thread) from a tree-owned counter and never returned, matching a model of a fixed pool of long-lived worker threads rather than one identity per short-lived task. Each thread remembers the identities it has already been given in thread-local storage, keyed not by the tree’s address but by a process-wide unique id assigned to the tree at construction — necessary because a dropped tree’s memory can be reused by an unrelated later tree, and a thread that cached an identity for the old occupant must not silently reuse it for the new one.

The cache itself keeps exactly two most-recently-used entries inline, in a plain copyable cell with no runtime borrow-checking, plus a lazily allocated overflow list for the rare case of a thread touching more than two trees. Two inline slots (not one, and not a larger searchable array) are a deliberate match to the actual access pattern: real deployments create exactly one tree that lives for the process’s whole lifetime, so the common case is a single, unchanging comparison against the primary slot; a thread that occasionally alternates between two trees still gets a cheap hit on the second slot; and only a thread that genuinely cycles through three or more trees pays for the overflow path at all. A larger fixed-size searchable array was tried first and discarded specifically because it optimized for a test-harness convenience (many short-lived trees per process, never interleaved) at the cost of complexity on every production lookup, where the actual shape of real usage is one or two long-lived trees.

10.2 The snapshot cache: structure-of-arrays visibility memoization

Visibility — “is this write, made by some other worker, visible to my snapshot” — is checked by comparing a reader’s start timestamp against a lower commit bound (LCB) computed from the writing worker’s own commit log. That LCB computation involves a lock and a search, so recomputing it on every single record a reader touches would be wasteful whenever the same foreign worker’s writes keep coming up against the same reader snapshot — exactly the common case inside one range scan or one busy transaction.

Each worker keeps its own cache, one entry per *other* worker it might need a visibility answer about, laid out as two parallel arrays rather than one array of pairs:

```
pub struct SnapshotCache {  
    // Structure-of-arrays: snapshot identity and LCB values are scanned or  
    // refreshed independently and no longer force 16-byte tuple traffic.  
    snapshot_versions: Box<[Version]>,  
    lcb: Box<[Version]>,  
}
```

The visibility check itself reads only what it needs, in an order chosen so the common cases touch the fewest cache lines:

```
if stamp.is_invalid() { return false; }           // aborted: never visible  
if stamp.worker_id() == reader_worker {  
    return stamp.ts_start() <= reader_ts_start;   // own writes: direct compare  
}  
let index = stamp.worker_id() as usize;  
if cache.lcb[index] > stamp.ts_start() {  
    return true;                                   // cache hit: touches `lcb` only  
}
```

```

}
if cache.snapshot_versions[index] < reader_ts_start {
    cache.lcb[index] = commit_logs[index].lcb(reader_ts_start);
    cache.snapshot_versions[index] = reader_ts_start; // refresh: locked, real query
}
cache.lcb[index] > stamp.ts_start()

```

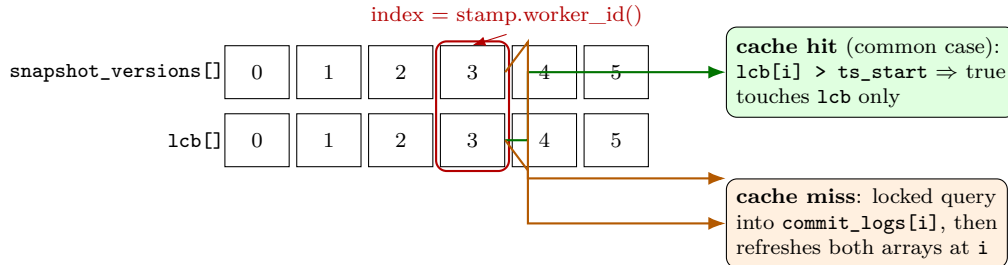


Figure 7: The snapshot cache’s two streams, indexed by the foreign worker’s id. A visibility check on the common, already-known-visible case reads only the `lcb` row; only a genuine miss reaches into the writing worker’s own commit log and refreshes both rows at that one index, leaving every other worker’s entries untouched.

An aborted write is rejected first, deliberately ahead of the same-worker fast path — otherwise that fast path would keep an aborted write visible to its own writer forever, since it never consults the commit log at all and an uncommitted write’s absence from that log would never get a chance to matter. A worker’s own writes are resolved with one direct comparison, no cache lookup needed, and deliberately support reading an intentionally historical snapshot (a start timestamp *older* than the write itself), not just “my current view.” For a foreign worker, the common case — this worker’s cached LCB for that foreign worker is already known to exceed the write’s timestamp — touches only the `lcb` array and returns immediately; only a genuine cache miss touches `snapshot_versions` and pays for a real, lock-protected query into that worker’s commit log. Because the two streams are separate arrays rather than one array of pairs, a hit never loads the `snapshot_versions` entry it does not need, and a miss’s refresh writes both without disturbing any other worker’s entries in either array.

This cache is created fresh for a tree’s actual configured worker budget, not against any fixed compile-time bound: sizing is a runtime parameter of the tree, one boxed slice per stream, sized once when the cache is created. There is no hard-coded maximum-worker ceiling baked into the type itself — a tree configured for more workers simply allocates larger boxed slices for its per-thread caches, and one configured for fewer wastes nothing on unused capacity.

Like the worker-identity cache above, this per-tree cache lives in thread-local storage, keyed the same way, with the same two-inline-slots-plus-overflow shape: the paper’s own “thread-local snapshot cache” idea is realized literally, using actual thread-local storage rather than a shared array indexed by worker id, so that no thread ever *could* touch another worker’s cache by mistake, not merely a convention that every call site happens to follow.

11 OSIC and the write-ahead log

11.1 Instant commit, lazily hardened

Every transaction draws its start and commit timestamps from one global, monotonically increasing counter. Each worker keeps its own append-only commit log; committing means drawing a commit timestamp and pushing it onto that log under a short lock — an in-memory step, with nothing on the commit path waiting on disk I/O. Durability catches up separately and in

batches: writes and commit markers are appended to a background writer, which periodically batches everything accumulated so far into one write-and-sync call and advances a published “hardened” watermark that a caller can poll explicitly if it specifically needs a durability guarantee. Recovery replays only writes that have a matching commit marker, ordered by commit time rather than log order, since commit order can differ from log order across concurrent workers.

11.2 Sharding the write channel

A single shared channel from every committing worker to one background flush thread is itself a piece of shared, contended state, in exactly the sense §9 describes: many independent writers funnel into the same channel and the same background thread. Splitting the channel into several independent (channel, flush-thread) shards, keyed by worker identity, removes that contention the same way the other sharding optimizations do — each shard still writes through one shared file handle, so the durable format and recovery logic are unaffected, but the hot enqueue path no longer funnels every worker through the same lock-free structure.

Workload	Threads	Before sharding	After sharding
Write-heavy OLTP (order-processing benchmark)	16	40,902 tx/s	43,913 tx/s
Write-heavy OLTP	64	57,667 tx/s	67,812 tx/s (+17.6%)
Write-heavy key-value, Zipfian	16	1,603,135 ops/s	2,010,940 ops/s
Write-heavy key-value, Zipfian	64	1,339,295 ops/s	2,258,954 ops/s (+68.7%)
Scan-heavy key-value	64	1,789,143 ops/s	2,679,747 ops/s (+49.8%)
Read-modify-write key-value	64	1,355,666 ops/s	2,305,318 ops/s (+70.1%)

Read-only and read-mostly workloads moved by under 1.2% either way, confirming the gain is specific to the write path: the two workloads that had previously gotten *worse* with more concurrency (the Zipfian and scan-heavy cases above) scale up with thread count instead of down, once this contention is removed.

11.3 A lock-free alternative backend and per-thread batching

A second design for the same write path removes the shared channel and dedicated flush thread entirely: each worker reserves its own byte range in the log file directly and writes into it itself, optionally batching its own records locally before flushing rather than issuing one write call per record. Both designs are available behind one interface, selected once when durability is attached.

Backend	@8 threads	@16 threads
Sharded channel (above)	1,006,755 ops/s	1,353,816 ops/s
Lock-free, unbatched (one write call per record)	752,179 ops/s	748,676 ops/s
Lock-free, batched locally per thread	2,549,030 ops/s	5,089,502 ops/s

Unbatched lock-free writes lose to the sharded channel past a handful of threads: each write still pays a full system call, and several threads extending the same growing file compete for the same filesystem-level bookkeeping. Batching each thread’s own records locally before flushing is what actually wins, amortizing that same system-call cost a different way than the sharded channel’s own batching does — across one thread’s own records instead of across many threads funneled through one flusher. §15 reproduces this comparison independently with a fuller thread and batch-size sweep.

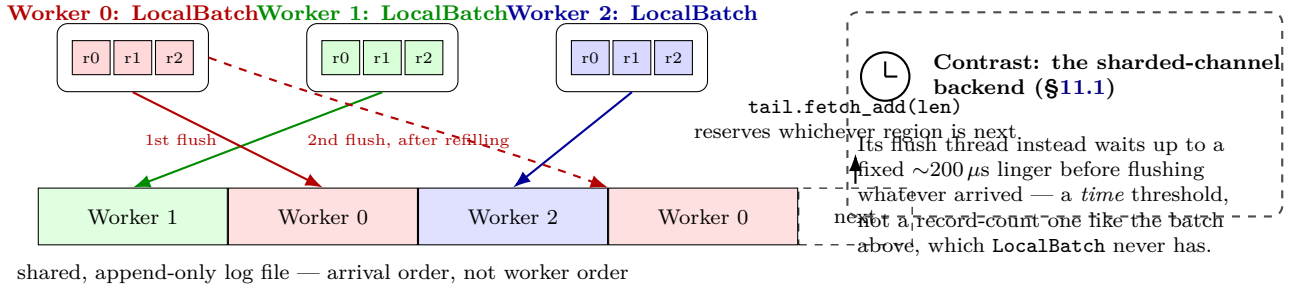


Figure 8: Each worker accumulates its own records in a thread-local batch, with no lock and no shared channel; only the flush at the end of a batch touches the shared file, as one `pwrite` into a byte range that worker alone reserved via one atomic `fetch_add` on the file’s tail. Reservations land in whatever order workers actually flush in — here, Worker 1, then Worker 0, then Worker 2, then Worker 0 again — never sorted or grouped by worker id, and never waiting for a turn: that is the actual benefit of a shared atomic counter over a shared lock or a round-robin protocol.

11.4 Ticket-less logging and exact framing sizes

Two further, purely allocation-side costs were found and removed once the contention above was no longer dominant. First, every log call built a fresh acknowledgment channel to hand back a “flush ticket” that ordinary production callers never actually used — logging methods now default to a ticket-less path with no channel built at all, with an explicit ticketed variant kept only for the handful of callers that genuinely need to wait on one specific record. Second, every framing buffer’s capacity hint had been sized for a small fixed-width test payload, forcing a real row’s much larger encoding to grow and reallocate the buffer several times during a single write; each payload type now reports its own exact encoded size up front, so the buffer for a real row is sized once, correctly, on the first allocation.

Metric	Before	After	Change
Write-heavy OLTP throughput	2,675,157 tx/min	3,013,193 tx/min	+12.6%
Write-heavy key-value ops/sec	1,374,921	1,723,455	+25.3%
Synthetic writer, 16 threads, small fixed payload	1,353,816 ops/s	2,427,071 ops/s	+79%

The synthetic case gains the most because, against a small fixed payload, the discarded ticket’s channel allocation was close to the entire per-write cost once the payload itself no longer needed resizing. An allocator that gives cross-thread frees a lock-free path (rather than routing a foreign thread’s free back through the allocating thread’s own arena) was tried as an opt-in alternative for the sharded channel’s cross-thread allocate/free pattern specifically; the effect measured out workload-dependent (a clear gain on one write-heavy benchmark, a clear loss on another), so it remains an optional feature rather than the default.

12 Range scans and analytical terminals

12.1 Cursor-based internal routing

An internal page is append/version ordered, not key ordered: structural modifications append replacement child entries while retaining historical entries that older snapshots still need to route through. A range scan therefore cannot walk physical slots left to right, or sort every snapshot-matching slot, without risking overlapping, superseded entries being included alongside their replacements. Instead, at each level the scan searches entries in reverse append order and selects the newest snapshot-visible interval containing the current scan cursor, descends, and

after consuming a leaf advances the cursor one key past that leaf’s boundary and reroutes from the retained parent — the same selection rule as a point lookup, allocating no child lists or deduplication structures at any point. An exact full-domain scan is detected once, before the leaf loop begins, and uses a loop that skips the two range-bound comparisons every bounded scan still needs per record, since a full-domain scan’s range trivially contains every key it could ever see.

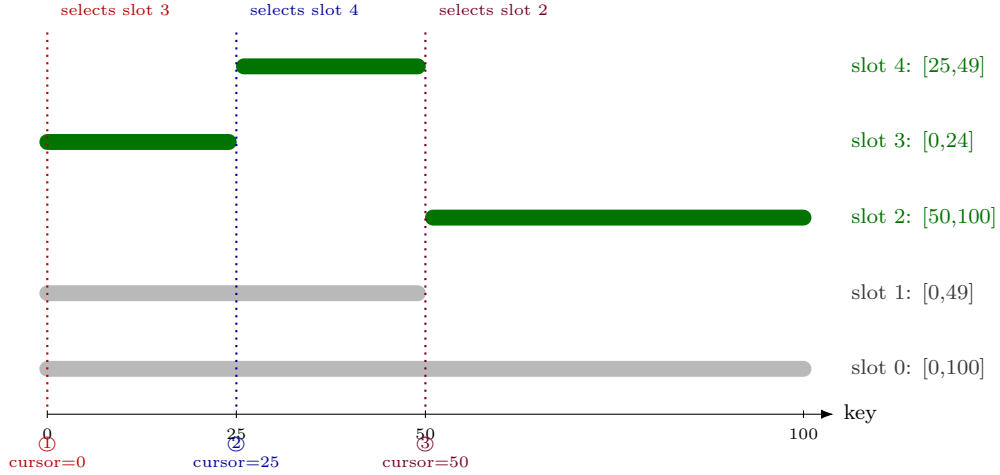


Figure 9: The same append-ordered partition as Figure 2, now followed by a scanning reader. At cursor 0, the newest entry whose interval contains it is slot 3 (slot 1 also contains key 0, but slot 3 is newer). After consuming slot 3’s leaf, the cursor advances to one past its fence (25) and re-selects at the same parent, landing on slot 4; advancing past slot 4’s fence (49) lands on slot 2. No child list is ever built — each step is one reverse scan of the parent’s entries, the same selection rule a point lookup already uses.

12.2 Zero-copy streaming terminals

Alongside the ordinary iterator (which materializes each visible record into an owned result and clones its payload handle, for callers who need results to outlive the scan), a family of terminal operations — for-each, fold, count, and a fallible early-exit variant — calls a visitor directly with a borrowed key and payload reference, never constructing an owned result object or cloning a payload handle at all. An analytical consumer that only ever aggregates (sums a column, counts matching rows) never needed an owned copy in the first place; these terminals let it not pay for one.

Measured: on a 200,000-row full scan, the streaming terminal ran $1.165\times$ – $1.173\times$ faster than the materialized iterator ($\sim 16.8\%$; 28.8–29.3 M rows/s vs. 24.7–25.1 M rows/s). A specialized counting terminal, which additionally skips constructing even a borrowed-result wrapper, reached ~ 49 – 53 M rows/s on the same kind of full scan.

12.3 Testing the cheap predicate first

Every physical record in a leaf is filtered by two predicates: is it within the requested key range, and is it visible to the reader’s snapshot. The visibility check can require up to two lookups into a foreign worker’s commit log (one for the insertion stamp, one for the deletion stamp, via the snapshot cache in §10); the range check is one or two integer comparisons. Ordering the range check first means an out-of-range record, however many superseded versions of other keys it happens to share a leaf with, is rejected before ever touching the more expensive check — a direct instance of the same “reject cheaply before paying for the expensive check” pattern the

leaf’s live/validity mask (§3) already applies one level earlier, applied here to the scan’s own per-record filter instead of to a whole slot. A full-table scan gets no benefit from this reordering (its range trivially contains everything), but a narrow scan sharing a leaf with many keys outside its own range benefits directly, in proportion to how many of that leaf’s records the range check alone can already reject.

12.4 Monomorphizing the visibility check

The reordering above only helps records the range check can already reject; every record actually within range still pays for the visibility check itself, and that check previously took its “is this visible” predicate as a type-erased callback — an indirect call the compiler cannot inline, paid on every insertion and deletion stamp of every in-range record. The range-scan iterator’s own visibility check now builds a concrete, statically-typed closure once per refill (covering every leaf that one refill call visits, not rebuilding it per leaf), so the compiler can inline the whole check directly into the scan loop rather than dispatching through a function pointer for every single record.

13 Workload and benchmark-harness optimizations

A number of optimizations sit in the benchmark and workload-generation code itself rather than in the storage engine proper — worth including in full, since an engine-side optimization measured against a benchmark that is itself spending unrelated CPU time on request generation will systematically understate its own effect.

13.1 Fast, thread-local randomness

The obvious choice for a random-number generator, `rand`’s default, is cryptographically secure (ChaCha-backed) — unnecessary strength for load-generator data that never needs to be unpredictable, and expensive enough that it measured as **up to ~14% of all CPU cycles** in a read-heavy benchmark, spent purely on picking which key to touch next. The fix follows the same shape everywhere it is applied: seed a fast, non-cryptographic generator once per thread from the secure generator, then draw from that thread-local instance for every subsequent request — paying the expensive seeding cost exactly once per thread’s lifetime, not once per operation:

```
thread_local! {
    static FAST_RNG: RefCell<SmallRng> = RefCell::new(rand::make_rng());
}
fn with_fast_rng<R>(f: impl FnOnce(&mut SmallRng) -> R) -> R {
    FAST_RNG.with(|rng| f(&mut rng.borrow_mut()))
}
```

The key-value benchmark driver uses `SmallRng` (Xoshiro256++); the order-processing benchmark driver uses `fastrand`’s `Rng` (WyRand) instead — two different concrete generators for two different drivers, but the identical pattern: a thread owns its own generator exclusively, with no shared state and no lock, the same principle behind every per-worker cache elsewhere in this document (§9, §10).

13.2 The alias-table Zipf sampler

A Zipfian-skewed workload needs to draw a “how popular is this key” rank from a skewed distribution on every single request. Computed naively, that draw needs a floating-point power evaluation per request — itself a real cost at millions of requests per second. Walker’s alias method avoids this by doing all of the expensive work exactly once, at setup: it builds a small

table of paired probabilities and alternate indices in $O(n)$ time (the only place a power computation ever runs), and after that, sampling is $O(1)$ — one random index draw, one random comparison, and a table lookup, with no floating-point power anywhere in the timed request path:

```
fn sample(&self, rng: &mut SmallRng) -> u64 {
    let column = rng.random_range(0..self.probability.len());
    let index = if rng.random::<f32>() < self.probability[column] {
        column
    } else {
        self.alias[column] as usize
    };
    index as u64 + 1
}
```

The table itself is immutable after construction, so every thread samples from the same one without needing a lock or a per-thread copy — only the source of randomness driving the two draws above is thread-local, per the previous subsection.

13.3 Rank scrambling: a realism fix, not a speedup

One subtlety is easy to get wrong here. Left alone, a Zipf sample’s rank is used directly as the key: rank 1 (the most popular draw) *is* key 1, rank 2 is key 2, and so on for the whole run. For this specific tree, that means the entire hot set of a Zipfian workload sits in the lowest few thousand keys for the run’s whole duration — physically, the same handful of leftmost leaf pages, every time. That is a windfall this engine did not earn: a page-based buffer-pool engine being compared against in the same benchmark has to keep a hot set that is *spread across the full key range* warm under a fixed memory budget, which is a meaningfully harder problem than keeping a few always-resident leftmost pages warm. Comparing raw throughput under that asymmetry would not be comparing the two engines’ actual concurrency/caching behavior at all.

The fix hashes the drawn rank through an FNV-1 hash before treating it as a key, matching the scrambling real Zipfian workload generators apply for exactly this reason. This preserves the skew — rank 1 is still drawn most often, and still resolves to one fixed key for the whole run — while spreading *which* physical keys are hot across the entire key range instead of clustering them at the bottom of it. This is deliberately *not* applied to a “read the most recently inserted keys” access pattern, whose entire point is bias toward physically recent keys — scrambling that would defeat the pattern it is meant to model. Unlike every other optimization in this document, this one does not make the engine faster: it makes the benchmark honest about what it is measuring.

13.4 Other workload-generation optimizations

Change	What it does, and why	Evidence
Bulk-generated values	Rows allocate their final payload once and fill it in place from buffered random bytes, instead of building and then copying a temporary value. Measured +23.5% on a write-heavy key-value benchmark.	Measured
Single-field update mode	A standard update patches one field via a leaf-latched, read-modify-write closure, rather than regenerating and replacing an entire row — the realistic case for most real update workloads. Measured +90.8% against the older full-row-replacement baseline (different semantics, not a like-for-like comparison).	Measured
Hit/miss-only reads	A read that only needs to know whether a key exists tests visibility in place and returns a boolean, with no result vector or payload clone. Combined with the alias sampler above, measured +106.8% on a read-heavy benchmark.	Measured
Sampled scan-latency measurement	Only one in every 1,024 scans is individually timestamped, and the shared timeseries clock is refreshed every 256 operations rather than on every one. Measured +35.4% on a scan-heavy benchmark.	Measured
Full-domain scan specialization	The exact full-key-space case is detected once and uses a visibility-only loop, described in §12.	included above

14 Case study: trading contention for scan cost

This section follows one optimization end to end because it is the clearest example in the system of a design whose two extremes are each bounded by a genuinely different, mechanistic cost — worth understanding as a whole rather than as a single tuning knob.

14.1 The problem: two tiny tables dominate contention

Attributing where concurrent transactions spend time waiting to retry finds that two of the smallest tables in a typical order-processing schema — tables with a handful of rows per warehouse-like partition — account for the overwhelming majority of all such retries, despite being tiny by row count. The reason is write volume, not size: every order touches them, their physical version chains grow quickly, and a leaf’s compaction threshold is a hard, fixed fraction of its capacity, so these two tables cross that threshold and force a page-locking compaction far more often, per row, than any other table in the schema.

14.2 The fix: an independent, tunable leaf capacity

These two tables are given their own tree configuration, independent of every other table’s leaf capacity, chosen from a small family of page-exact sizes rather than one hand-picked number. Giving *only* these two tables a larger capacity avoids paying the memory and alignment cost

of a larger leaf on every one of the schema’s other tables, which were never the source of the contention in the first place.

14.3 Why the trade-off has a floor and a ceiling

The upper end of this trade-off is unavoidable, not a tuning oversight: a leaf’s compaction threshold is also exactly when its accumulated dead, superseded versions get reclaimed, and a range scan over that table has to examine a leaf’s entire physical record array, live and dead alike. A larger capacity therefore means more accumulated garbage sits in the leaf between compactions, and every scan over that table pays for wading through it — there is no way to independently choose “avoid frequent compactions” and “keep scan-time garbage bounded” in this design; they are the same knob, turned in opposite directions.

The lower end has a genuine floor for a structural reason: every split pushes two fresh entries into a parent rather than overwriting one, so a freshly split node needs enough raw capacity to accept at least one further push before it, too, needs to split — push a capacity small enough and a freshly split node can find itself already back at its own overflow threshold with no room to accept even the very next write, regardless of how much concurrency is involved. This makes the smallest usable capacity a property of the split algorithm itself, not merely something that happens to perform poorly if chosen too small.

14.4 The measured middle ground

The default choice sits at the point where a scan over these two tables’ rows recovers essentially full throughput relative to an unmodified leaf, while still capturing most of the achievable reduction in retry contention; larger capacities trade further contention headroom for scan cost that grows faster than the contention benefit does, once the scan has to age past the first few seconds of accumulated garbage. This is a genuinely explicit, chosen point on a continuum, not a default that happens to work: reaching for a larger capacity is a deliberate choice for a deployment whose read pattern can tolerate slower scans on these two tables in exchange for less contention on their writes, not a strictly better setting.

15 Fresh measurements, independently reproduced

Two of the microbenchmarks above were re-run, independently, while writing this document, specifically to check the historical numbers against the system as it exists today rather than simply restate them.

15.1 Leaf layout, reproduced on different hardware

Following the same protocol as §3’s original measurement (a standalone, dependency-free binary, five repeated runs, pinned to one core), the 62.5 MiB multi-leaf sweep was re-run on different, older hardware than the original report:

Operation	Array-of-structs	Structure-of-arrays	Speedup
Point: latest visible	10.90 Mops/s	11.27 Mops/s	1.033×
Point: 1-version fallback	7.86 Mops/s	8.43 Mops/s	1.073×
Point: 3-version fallback	4.66 Mops/s	4.32 Mops/s	0.927×
Scan: full leaves	245.2 Mslots/s	236.2 Mslots/s	0.963×
Scan: 20% ranges	254.0 Mslots/s	236.0 Mslots/s	0.929×

The five runs agreed with each other to well under 1%, so this is not measurement noise: it is a genuine, reproducible result that nonetheless diverges from the original report, both in

magnitude (roughly a third of the claimed point-lookup gain) and, for the 3-version and scan cases, in *direction*. The most plausible explanation is the hardware itself: the split layout’s benefit is specifically a cache-pressure effect (fewer unrelated cache lines touched per rejected key), so an older CPU with a smaller, higher-latency last-level cache would be expected to show a smaller version of the same effect, and could plausibly cross zero once the layout-independent cost of touching every byte (as a full scan always must) already dominates. This divergence is reported in full rather than smoothed over, since a microbenchmark result that depends on the specific cache hierarchy it ran on is itself a useful thing to know about the benchmark, not just about the layout.

15.2 Write-ahead-log backend throughput, reproduced with a fuller sweep

The same synthetic write-path comparison from §11 was re-run in an optimized release build, across a fuller thread sweep and batch-size sweep than the original report used:

Backend	@1 thread	@8 threads	@16 threads
Sharded channel	700,592 ops/s	2,607,583 ops/s	4,339,121 ops/s
Lock-free, unbatched	384,583 ops/s	839,536 ops/s	763,438 ops/s
Lock-free, batch of 16	1,509,518 ops/s	4,898,065 ops/s	5,563,703 ops/s
Lock-free, batch of 64	1,475,161 ops/s	5,979,338 ops/s	7,578,144 ops/s
Lock-free, batch of 256	1,515,828 ops/s	6,386,070 ops/s	8,678,745 ops/s

This reproduces the qualitative ranking from §11 cleanly: unbatched lock-free writes lose to the sharded channel past a handful of threads, and per-thread batching is what actually wins, improving monotonically with batch size. The absolute numbers are considerably higher than the original report’s (built without the same optimization settings), which is expected and not itself meaningful — the ranking between backends, not the absolute numbers, is what the original conclusion rested on, and it reproduced exactly.

16 Combined measured gains at a glance

This table collects only controlled, quantitative results from the sections above. It is the right table to cite when a specific number is needed; a row measuring a combined batch of changes should not be attributed to one component of that batch in isolation.

Optimization	Workload	Measured change
Exact page-sized leaf capacity	Allocation / TLB profile	Alignment 25%→100%; dTLB load misses -9 to -12%
Structure-of-arrays leaf layout	Point lookup, multi-leaf	1.09–1.22× (original hardware)
Sharded WAL write channel	Write-heavy OLTP, 64 threads	+17.6%
Sharded WAL write channel	Write-heavy key-value, 64 threads	+68.7%
Sharded WAL write channel	Scan-heavy key-value, 64 threads	+49.8%
Sharded WAL write channel	Read-modify-write, 64 threads	+70.1%
Ticket-less logging + exact framing sizes	Write-heavy OLTP	+12.6%
Ticket-less logging + exact framing sizes	Write-heavy key-value	+25.3%
Ticket-less logging + exact framing sizes	Synthetic writer, small payload	+79%
Lock-free backend, local batching	Synthetic writer, 16 threads	5.09–8.68M vs. 0.75–1.35M ops/s unbatched
Zero-copy streaming scan terminal	Full-table scan	+16.8%
Range-before-visibility ordering	Narrow per-key sub-scans	+11–18%
Bulk value generation + single-field updates	Write-heavy key-value	+23.5% / +90.8%
Alias-table sampling + hit-only reads	Read-heavy key-value	+106.8%
Sampled scan-latency measurement	Scan-heavy key-value	+35.4%
Independent leaf-layout reproduction	Point lookup, different hardware	1.03–1.07× (smaller than original)

17 Conclusion

Taken together, the optimizations in this document are variations on a small set of ideas, applied at every layer of the system: lay data out so a cheap check can reject most of it before a more expensive one runs (the leaf’s live mask, the internal page’s live mask, range-before-visibility ordering); give every lock-free reader something it can validate without ever taking a lock (optimistic lock coupling, the snapshot cache); and partition shared bookkeeping along the one axis — worker identity — that lets each partition be touched by only the thread that owns it (per-worker snapshot slots, sharded reclamation, sharded write channels). The version-growth bound in §7.1 and the root-caching and batched-abort optimizations in §7 are the same idea again, one layer up: notice what a transaction is guaranteed never to need to repeat, and stop paying for repeating it.

References

- [1] Becker, B., Gschwind, S., Ohler, T., Seeger, B., Widmayer, P. *An asymptotically optimal multiversion B-tree*. The VLDB Journal, 5(4):264–275, 1996.
- [2] Tonta, A., Seeger, B., Soisalon-Soininen, E. *Multiversion Concurrency Control for Multiversion B-Trees*. arXiv preprint arXiv:2606.09133, 2026.
- [3] Alhomssi, A., Leis, V. *Scalable and Robust Snapshot Isolation for High-Performance Storage Engines*. Proceedings of the VLDB Endowment, 16(6):1426–1438, 2023. doi:10.14778/3583140.3583157.